Unit 2 Refined Backlog – RigAssist AI

Macauley Wain – Proposed Group 3 Review

My Recommendation

After reviewing the initial Copilot-generated backlog and comparing it with what we are ultimately expected to deliver in Unit 8, I recommend that we simplify the scope of RigAssist AI. I do not think we need to change the core idea. Instead, I think we should reduce the number of supporting systems we try to build so we can spend more time on integration, testing, CI/CD, documentation, performance evidence, and a reliable final demonstration.

My proposed MVP keeps three core capabilities: a deterministic rigging calculator written in Swift, a small read-only local reference library, and a constrained AI assistant that explains supported concepts using approved reference information. The AI would not perform or override safety-critical calculations.

Three AI-Generated Tasks I Recommend Refining

1. RA-006 – Full Local CRUD Data Model

Classification: Out of scope

My reasoning: The original backlog treated the local reference information like a database that needed full create, read, update, and delete functionality. I do not think our prototype needs users to manage rigging records. We only need to retrieve a limited set of approved sample information.

Proposed revision: Replace this with a small read-only local JSON dataset containing supported sample reference records and source metadata.

2. RA-022 – Full RAG/Retrieval Layer

Classification: Technically risky

My reasoning: A full retrieval-augmented generation or vector-retrieval implementation could become a separate project by itself. It adds retrieval quality, embedding/search strategy, integration, testing, and additional dependencies that are not necessary to prove our concept.

Proposed revision: Use a constrained reference service that retrieves relevant records from our small approved dataset and supplies that context to the AI assistant.

3. RA-026 – Guarantee Grounded AI Responses

Classification: Technically risky

My reasoning: The intent is good, but guaranteeing that an LLM only produces grounded statements is a difficult standard for an eight-week student prototype. I think we should demonstrate controlled behavior without claiming that the model can never generate an unsupported statement.

Proposed revision: Limit the AI to supported questions, provide approved reference context, display source information, and use a clear fallback response when the question cannot be supported.

Refined Week 1–4 Sprint Backlog

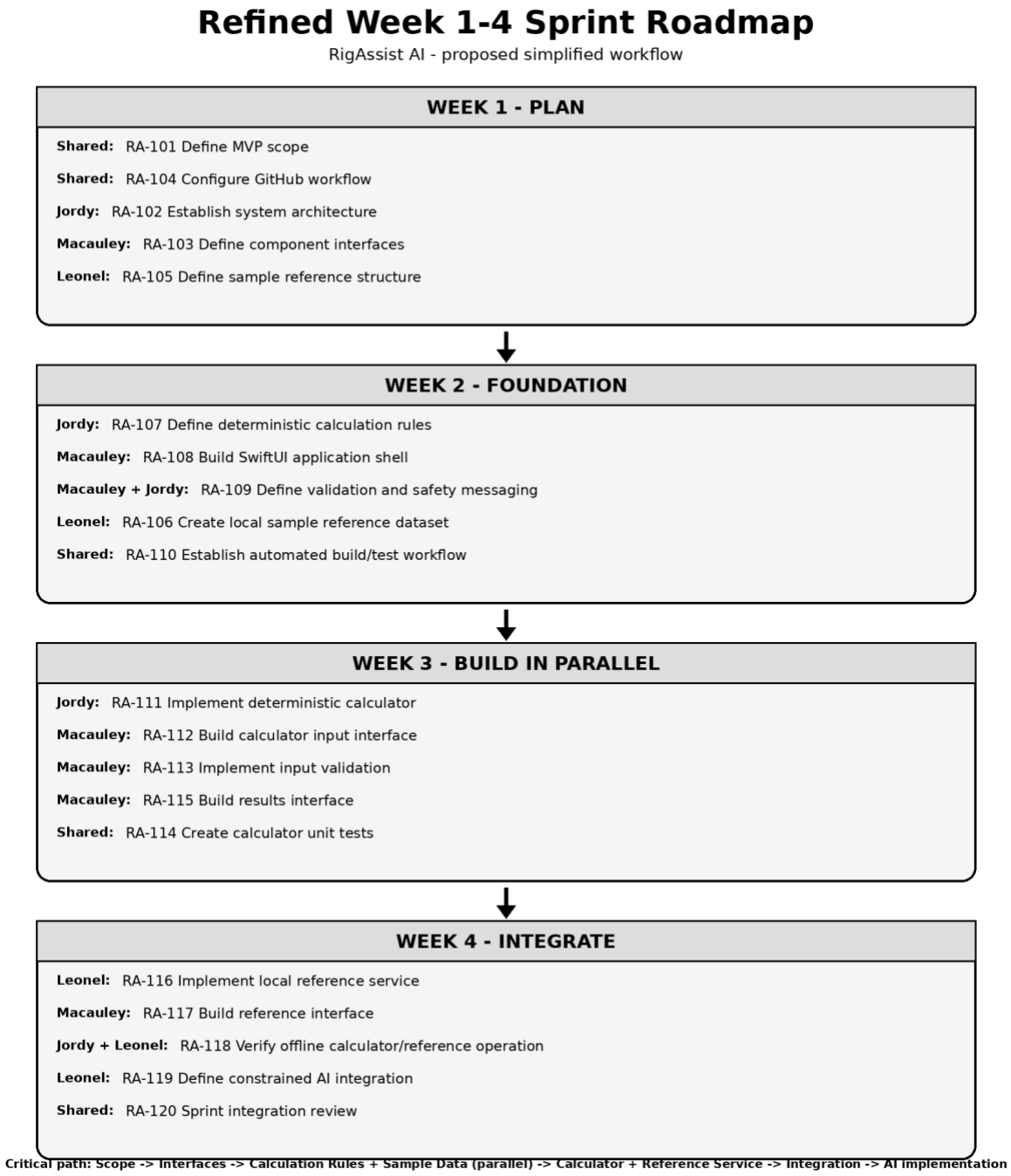


Figure 1. Proposed refined Week 1-4 sprint workflow. Tasks are grouped by week and show primary ownership; several implementation tracks can proceed in parallel.

RA-101 – Week 1: Define MVP scope

Type: Research/Design **Owner:** Shared **Effort:** Small

Dependencies: None

Acceptance Criteria: Team agrees on supported inputs, configurations, calculator functions, reference scope, and AI boundaries.

RA-102 – Week 1: Establish system architecture

Type: Research/Design **Owner:** Jordy **Effort:** Medium

Dependencies:RA-101

Acceptance Criteria: Architecture defines SwiftUI Interface, Calculation Engine, Reference Service, and AI Service.

RA-103 – Week 1: Define component interfaces

Type: Research/Design **Owner:** Macauley **Effort:** Medium

Dependencies:RA-101

Acceptance Criteria: Inputs, outputs, and interactions between the four modules are documented.

RA-104 – Week 1: Configure GitHub workflow

Type: NFR / Technical **Owner:** Shared **Effort:** Small

Dependencies: None

Acceptance Criteria: Repository, branches, issues, contribution expectations, and initial README are operational.

RA-105 – Week 1: Define sample reference structure

Type: Research/Design **Owner:** Leonel **Effort:** Small

Dependencies:RA-101

Acceptance Criteria: Local data structure supports approved sample records and source metadata.

RA-106 – Week 2: Create local sample reference dataset

Type: FR **Owner:** Leonel **Effort:** Medium

Dependencies:RA-105

Acceptance Criteria: App-readable dataset contains agreed sample records with source metadata.

RA-107 – Week 2: Define deterministic calculation rules

Type: Research/Design **Owner:** Jordy **Effort:** Medium

Dependencies:RA-101

Acceptance Criteria: Supported calculations, required inputs, outputs, and unsupported conditions are documented.

RA-108 – Week 2: Build SwiftUI application shell

Type: FR **Owner:** Macauley **Effort:** Medium

Dependencies:RA-103

Acceptance Criteria: App launches with navigation for Calculator, Reference, and AI Assistant views.

RA-109 – Week 2: Define validation and safety messaging

Type: NFR **Owner:** Macauley / Jordy **Effort:** Small

Dependencies:RA-103, RA-107

Acceptance Criteria: Invalid or missing inputs produce understandable errors and prototype limitations are visible.

RA-110 – Week 2: Establish automated build/test workflow

Type: NFR / Technical **Owner:** Shared **Effort:** Medium

Dependencies:RA-104

Acceptance Criteria: Repository automatically runs available tests on designated pushes or pull requests.

RA-111 – Week 3: Implement deterministic calculator

Type: FR **Owner:** Jordy **Effort:** Medium

Dependencies:RA-107

Acceptance Criteria: Supported inputs produce expected results independently of the AI system.

RA-112 – Week 3: Build calculator input interface

Type: FR **Owner:** Macauley **Effort:** Medium

Dependencies:RA-108, RA-107

Acceptance Criteria: User can enter or select all supported calculator variables.

RA-113 – Week 3: Implement input validation

Type: FR / NFR **Owner:** Macauley **Effort:** Medium

Dependencies:RA-109, RA-112

Acceptance Criteria: Missing, invalid, and unsupported inputs are rejected before calculation.

RA-114 – Week 3: Create calculator unit tests

Type: Testing **Owner:** Shared **Effort:** Medium

Dependencies:RA-111

Acceptance Criteria: Known supported scenarios pass automated tests.

RA-115 – Week 3: Build results interface

Type: FR **Owner:** Macauley **Effort:** Medium

Dependencies:RA-111, RA-112

Acceptance Criteria: User can view calculation results, relevant inputs, and verification messaging.

RA-116 – Week 4: Implement local reference service

Type: FR **Owner:** Leonel **Effort:** Medium

Dependencies:RA-106

Acceptance Criteria: Application retrieves appropriate records from the bundled sample dataset.

RA-117 – Week 4: Build reference interface

Type: FR **Owner:** Macauley **Effort:** Medium

Dependencies:RA-108, RA-116

Acceptance Criteria: User can view supported reference information and its source.

RA-118 – Week 4: Verify offline calculator/reference operation

Type: NFR **Owner:** Jordy / Leonel **Effort:** Small

Dependencies:RA-111, RA-116

Acceptance Criteria: Calculator and bundled reference information function without network connectivity.

RA-119 – Week 4: Define constrained AI integration

Type: AI FR / Design **Owner:** Leonel **Effort:** Medium

Dependencies:RA-103, RA-116

Acceptance Criteria:AI contract defines question, approved context, response, source information, and fallback behavior.

RA-120 – Week 4: Sprint integration review

Type: Testing / Documentation **Owner:** Shared **Effort:** Medium

Dependencies:RA-111–RA-119

Acceptance Criteria: Calculator, UI, local data, tests, and AI interface contract integrate without critical blockers.

Required AI User Story

As a field user, I want to ask RigAssist AI a question about a supported rigging concept so that I can receive an understandable explanation based on the application's approved reference information without relying on the AI to perform safety-critical calculations.

Acceptance criteria: The AI receives relevant approved context from the Reference Service; supported questions receive an explanation with source information; unsupported questions receive a clear fallback response; and the AI does not perform or override deterministic calculations.

Functional Requirements

- FR-1: Users can enter supported lift variables through the mobile interface.
- FR-2: Users can select from a limited set of supported sling configurations.
- FR-3: The application performs supported calculations using deterministic Swift code.
- FR-4: The application validates supported inputs before calculations execute.
- FR-5: The application retrieves approved sample reference information from a local read-only dataset.
- FR-6: Users can view reference information and its source.
- FR-7: Users can submit supported natural-language questions to the AI assistant.
- FR-8: The AI assistant receives approved reference context before generating supported explanations.
- FR-9: Unsupported AI questions return a clear fallback response.

Non-Functional Requirements

- NFR-1: Safety-critical calculations remain independent of the AI subsystem.
- NFR-2: The deterministic calculator and local reference information function offline.
- NFR-3: Automated tests verify known calculator scenarios.
- NFR-4: Validation and error messages are understandable to the user.
- NFR-5: Component interfaces are documented to support asynchronous development.
- NFR-6: Available automated tests run through the team's CI workflow.
- NFR-7: Prototype limitations and human-verification requirements are clearly communicated.
- NFR-8: Performance and reliability measurements are collected before final delivery.

How I Think the Work Should Be Divided

Jordy – Lead Architect

- System architecture and diagrams
- Deterministic calculation rules and calculator implementation
- Architecture/calculation documentation
- Calculation and offline verification

Macauley – Interface Designer

- Component interfaces and service contracts
- SwiftUI application shell
- Calculator inputs, validation, and results interface
- Reference and AI interaction interfaces
- User-facing source and safety messaging

Leonel – Integration Lead

- Sample reference structure and local dataset
- Reference-service implementation
- AI service contract and integration
- AI fallback behavior and evaluation approach

Shared

- MVP scope
- GitHub workflow and issues
- CI/CD setup
- Automated testing and code review
- Sprint integration reviews
- Final documentation and presentation work

Revised Critical Path and Workflow

I recommend avoiding the long linear critical path from the original AI-generated backlog. My proposed critical path is:

MVP Scope → Component Contracts → Calculation Rules and Sample Dataset (in parallel) → Calculator and Reference Service (in parallel) → Week 4 Integration → AI Implementation in the next sprint.

The SwiftUI interface, GitHub/CI setup, documentation, and testing can progress alongside these tasks. I think this is a better fit for our team because it reduces blocking and allows us to work asynchronously despite the time-zone difference.

Why I Think This Is the Better Approach

My goal with this revision is not to make RigAssist AI less capable. It is to make the project small enough that we can deliver the core features at a higher quality. The final project expects a functional integrated system, meaningful AI integration, CI/CD evidence, testing and performance metrics, professional documentation, code reviews, version-control evidence, and a stakeholder demonstration. Every extra feature we add also becomes another feature we have to integrate, test, document, benchmark, and explain.

I think a reliable calculator, a small approved reference library, and a constrained AI assistant give us enough technical depth to meet the project goals while leaving enough time to produce the quality evidence the final rubric expects. I also think establishing GitHub and our contribution workflow early will help us document our actual work as it happens rather than trying to recreate collaboration evidence near the end of the course.

This is my proposed starting point, and I would like Jordy and Leonel to review the ownership, effort estimates, and scope before we treat it as our final Unit 2 team backlog.